

# Automating AI-ML models to SNNs

~ Ananya Pawar & Hari Nisanth S

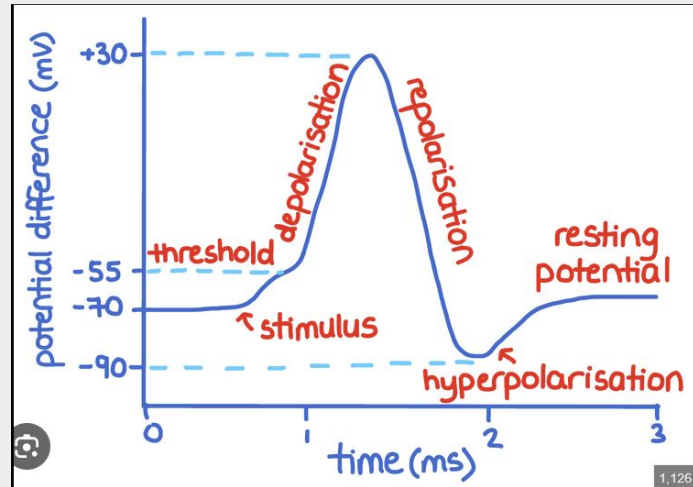
# Entering Spiking Neural Networks (SNNs)

- Spiking Neural Networks are closer to how real neurons work.
- Neurons communicate using **spikes (0 or 1)**
- Computation happens **only when spikes occur**
- Information is encoded in:
  1. spike timing (temporal)
  2. spike rate
- As discussed, we will not use activation functions, instead we use spiking neurons. Different neuron models introduce **different spike behaviors**. There are mainly 3 models : Leaky-Integrate Fire, Izhikevich Neuron, Hodgkin-Huxley.
- We will use the LIF model since it is easier to execute on hardware as it contains simpler and linear differential equation. Hodgkin-Huxley is biologically more closer to actual neuron behaviour but it is difficult to implement on hardware.

\*Source : google

# Leaky Integrate and Fire

- Imagine a bucket that collects water drop by drop, we transfer the water once it is filled upto the brim/overflows.
- Similarly, membrane potential keeps increasing since the neuron is seen analogous to an RC Circuit. Once the voltage exceeds a certain threshold, the neuron is said to be “fired” and then potential resets back to zero.



$$\tau_m \frac{dV(t)}{dt} = -[V(t) - V_{rest}] + R_m I(t)$$

LIF Membrane Potential equation

\*source : Google

# SCNNs vs Spiking Fully Connected Neural Network

- **Spiking Fully Connected Neural Network (S-FCNN):**  
Every neuron in one layer is connected to **all neurons** in the previous layer. It works on **spike inputs**, but the structure is dense.
- **Spiking Convolutional Neural Network (SCNN):**  
Uses **convolution (filters/kernels)** with spiking neurons to extract features, especially from **images and spatial data**.
- SCNNs use local receptive fields and weight sharing, making SCNNs more efficient and better suited for image-based tasks.

| Feature              | ANN (Artificial Neural Network)               | SNN (Spiking Neural Network)                         |
|----------------------|-----------------------------------------------|------------------------------------------------------|
| Data Representation  | Uses continuous real-valued activations       | Uses discrete spike events (0 or 1) over time        |
| Information Encoding | Amplitude-based (value magnitude matters)     | Time-based (spike timing and rate carry information) |
| Neuron Model         | Simple mathematical functions (ReLU, Sigmoid) | Biologically inspired models (LIF, Izhikevich)       |
| Computation Style    | Synchronous and layer-wise                    | Event-driven and time-dependent                      |
| Power Consumption    | Higher due to continuous computation          | Lower because computation happens only on spikes     |

|                     |                                               |                                                                                                          |
|---------------------|-----------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Hardware Efficiency | Optimized for GPUs and CPUs                   | Efficient on neuromorphic chips( ex- Intel Loihi) and FPGAs                                              |
| Training Method     | Backpropagation with gradient descent         | Backpropagation with surrogate gradients, STDP(spike timing dependent plasticity), ANN-to-SNN conversion |
| Latency             | Low inference latency                         | May need multiple time steps                                                                             |
| Memory Usage        | Stores full precision weights and activations | Stores binary spikes and simpler neuron states                                                           |
| Biological Realism  | Low                                           | High (closer to real brain behavior)                                                                     |

### **STDP:**

A biologically inspired learning rule where weights are updated based on the timing difference between pre-synaptic and post-synaptic spikes.

- If input spike comes before output spike → strengthen weight
- If input spike comes after output spike → weaken weight

### **Surrogate gradient ( Backpropagation):**

During training, we replace the spike function with a smooth fake function to compute gradients.

- Forward pass: still uses real spikes
- Backward pass: uses approximate gradient

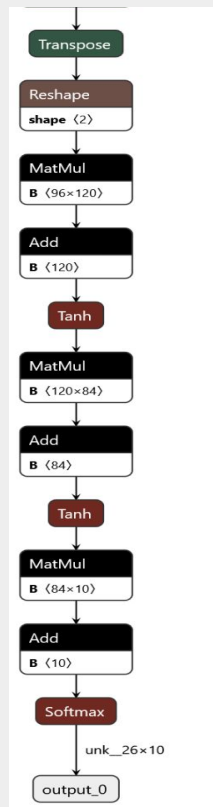
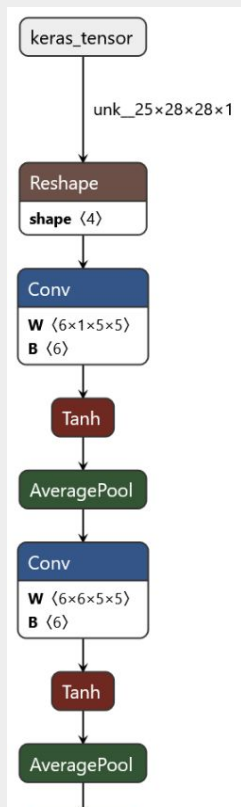
### **ANN-to-SNN Conversion**

#### **Idea:**

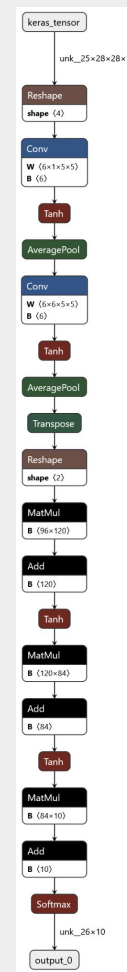
- Train a normal ANN **first** using backpropagation
- Then **convert** it to SNN by replacing ReLU neurons with spiking neurons and weights remain the same

The SNN will **approximate the ANN's behavior using spike rates.**

# Interpretation using Netron (ONNX)



Combined



# NengoDL

- Train neuromorphic models using deep learning methods.

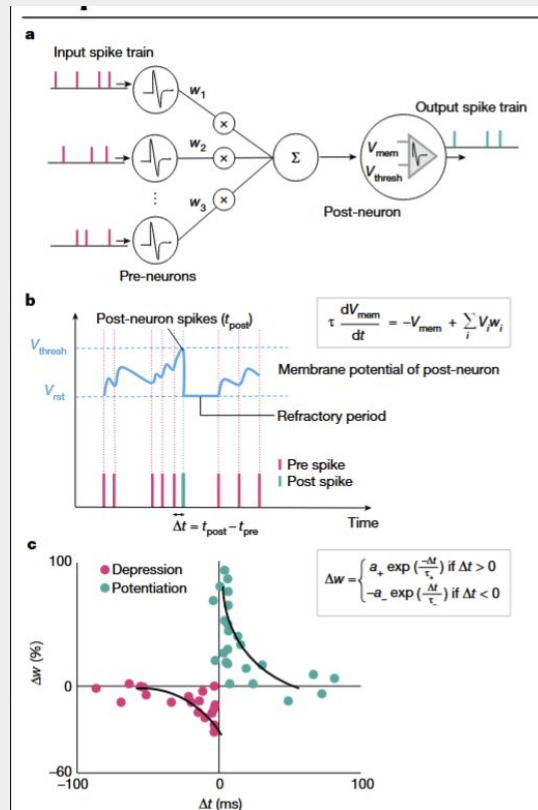
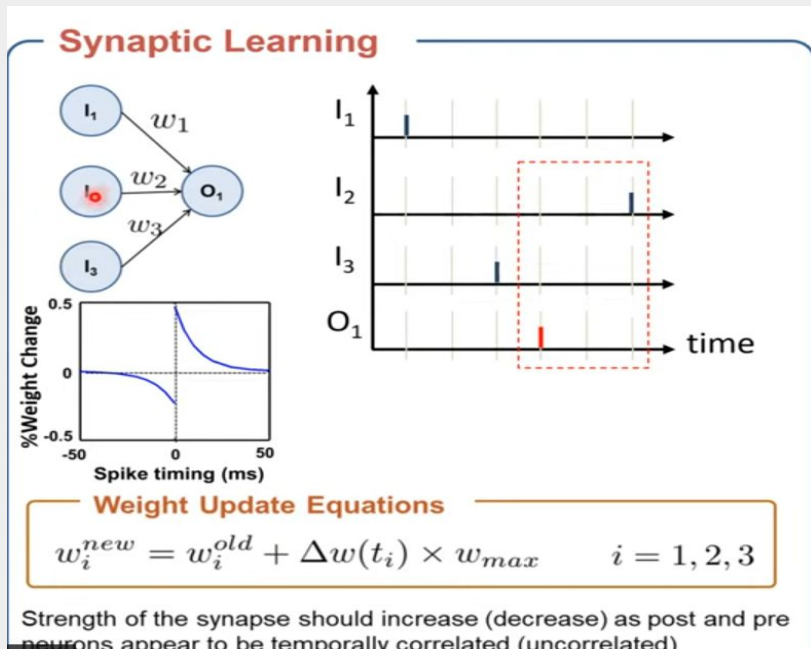
Some terms :

- A population of neurons representing some value (100 LIF neurons representing 1 variable) – **Ensemble**
- A **Node** is used to feed input into the network or output a fixed function
- **Connection** = weights between things. connects Node → Ensemble, Ensemble → Ensemble, Ensemble → Node. (like FC layer)
- **Probe** : records result

# SNN ToolBox vs NengoDL

- Toolbox only converts a trained ANN to SNN.
- No training happens after it is converted.
- NengoDL helps build and train model in NengoDL itself.
- Once converted, SNN can also be trained.
- Backpropagation also possible in NengoDL unlike toolbox.
- NengoDL helps us build Neuromorphic Models (like SNN) using deep learning tools (like tensorflow)

# STDP-



# Batch normalisation

- Enables fast training in ann
- Can we apply Batch Normalisation in snn?  
Yes. By using BNTT ( Batch normalisation through time)
- Gives very low latency
- Transforms the input by using mean and variance , then creates a new layer called BN layer which has learnable parameter gamma and beta, which helps in better training

# Batch Normalization Through Time (BNTT)

## Related work: Batch Normalization

- Reducing the internal covariate shift caused by the distribution change of input
- Normalization using batch-wise mean and variance
- To improve the representation capability of the layer, learnable parameters  $\gamma$  and  $\beta$  are used to transform

$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_{i=1}^m x_i; \quad \sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2$$

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

$$BN(x_i) = \gamma \hat{x}_i + \beta$$

- BNTT

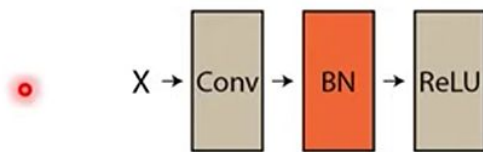
- Varying the internal parameters in a BN layer through time

- Standard LIF neuron

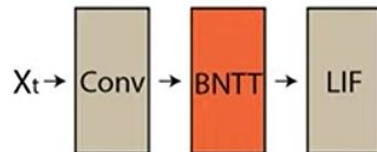
$$u_i^t = \lambda u_i^{t-1} + \sum_j w_{ij} o_j^t$$

- LIF neuron with BNTT

$$\begin{aligned} u_i^t &= \lambda u_i^{t-1} + BNTT_{\gamma^t} \left( \sum_j w_{ij} o_j^t \right) \\ &= \lambda u_i^{t-1} + \gamma_i^t \left( \frac{\sum_j w_{ij} o_j^t - \mu_i^t}{\sqrt{(\sigma_i^t)^2 + \epsilon}} \right) \end{aligned}$$



ANN (BN)



SNN (BNTT)

## Temporal early exit (BNTT benefit)

- In BNTT, if we train for 25 timesteps then the gamma becomes very low after 20 timesteps, so we can stop inference at 20 timesteps, as neurons don't spike much after that due to low gamma

BNTT is the best as it reduces latency for inference (which was the main problem for SNN)

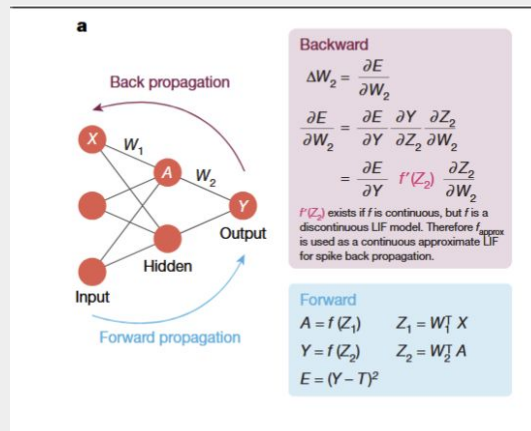
# Fast Sigmoid approximation for backward propagation (training SNN)

For gradient descent to work we have to have a differentiable function

LIF is non differentiable, so we use this function for back propagation in SNN

A smooth surrogate function is used during backpropagation to approximate the gradient.

(this is for training SNNs )



$$S(x) = \frac{x}{1 + |x|}$$

$$\frac{dS}{dx} = \frac{1}{(1 + |x|)^2}$$

# Input Encoding Methods

**Rate encoding**- Higher pixel value has More number of spikes

Done using poisson encoding , spikes are generated randomly, we don't have control over when the spikes are generated, we just know the probability of generation

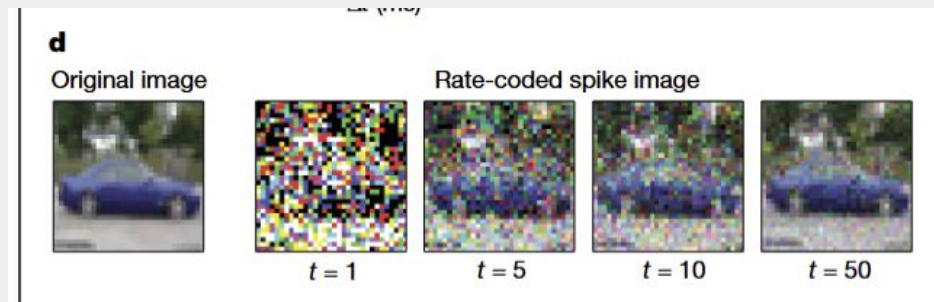
**Temporal encoding**- First occurring spike has larger pixel value

ie - larger pixel value spikes first

# Why we need higher timesteps

As no of time steps increases the image we feed to snn becomes more closer to original image , so higher timesteps resembles original image better as we are using poisson encoding for input spike generation which treats the pixel intensity as the probability of spike generation.

Lower Timesteps result in a lot of noise. As seen in the picture below : higher the time steps, cleaner is the image obtained since we have **spike rate ~ activation function**



# Synaptic Filtering

- Spikes are discrete and noisy. Hence they are smoothened into a continuous curve using :

$$h(t) = \frac{1}{\tau} e^{-t/\tau}$$

- Hence the current now becomes :

$$I(t) = \sum w_i (S_i * h)(t)$$

This smoothenes the spikes and approximates continuous ann activations

# How nengodl converts ann to snn

- Trains ANN normally (ReLU or rate neurons (LIFrate))
- Keep the learned weights (weights remain fixed now)
- Replace activations with **spiking neurons (LIF)**
- Convert inputs to **spike trains** (hidden layers **generate spikes dynamically** by integrating incoming spikes)
- Run network for **T timesteps** (increase T so that **spike rate ~ activation value**)
- Compute output using **spike rate**

IF neuron leak value= 0

$$V_{\text{mem}}(t+1) = V_{\text{mem}}(t) + \text{MAC fn}$$

If leak value was 5%

$$V_{\text{mem}}(t+1) = 0.95 * V_{\text{mem}}(t) + \text{MAC fn}$$

Backprop-trained SNNs learn sparse spike representations during training, which reduces the total number of spikes and timesteps required for inference, leading to faster and more energy-efficient computation compared to ANN-to-SNN converted networks.

# Result

When we trained a ANN and converted it to SNN using NengoDL, we observed the following results-

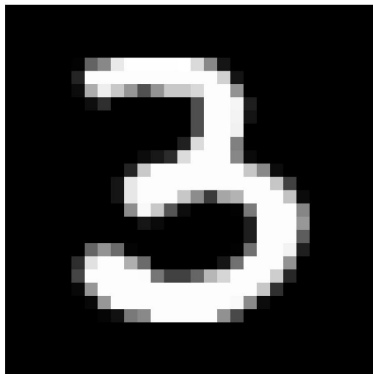
**ANN Accuracy:** 98.41%

**SNN Accuracy:** 98.33%

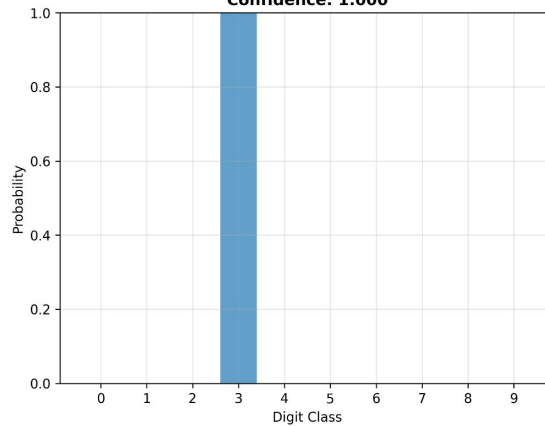
Main Code for conversion

```
converter = nengo_dl.Converter(  
    ann_model,  
    swap_activations={tf.keras.activations.relu: nengo.LIF}  
)  
#Converts the trained ANN model into an equivalent SNN and transfers its learned weights.
```

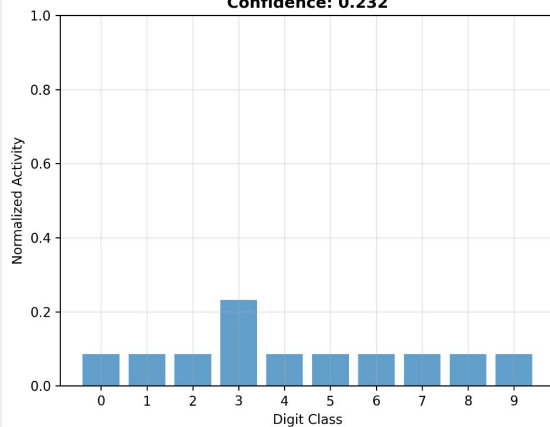
**Input Image**  
**True Label: 3**



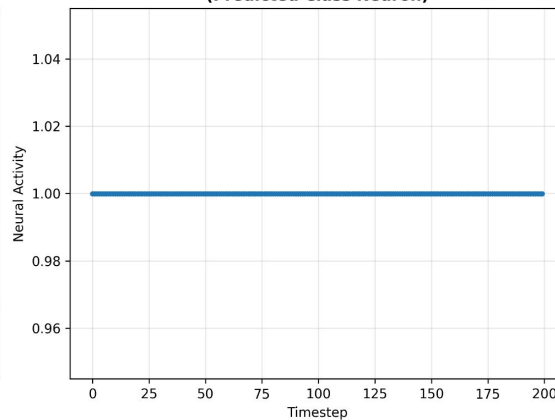
**ANN Prediction: 3**  
**Confidence: 1.000**



**SNN Prediction: 3**  
**Confidence: 0.232**



**SNN Activity Over Time**  
**(Predicted Class Neuron)**



How are weights transferred

$$I(t) = w \cdot \text{spike}(t)$$

$$V(t) = V(t - dt) + I(t) - \text{Leakage}$$

# More on weights

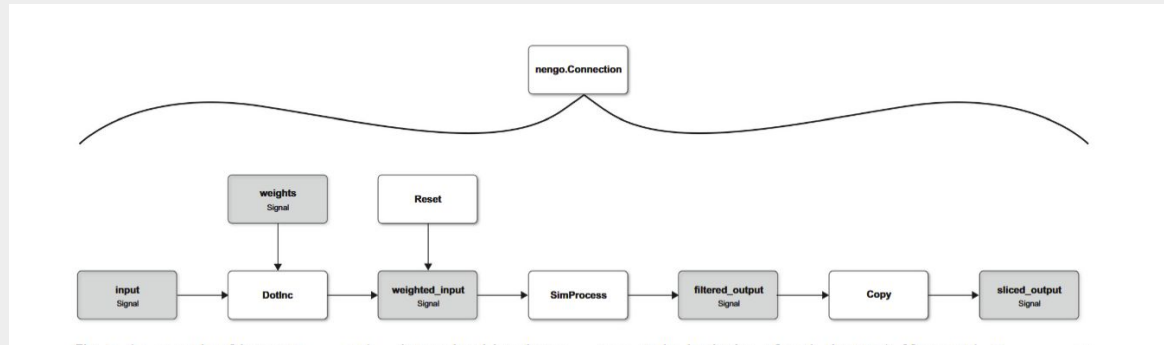
- Weights are not stored in binary, they are floating points and fixed after conversion.
- They exactly match the ANN weights obtained after training.
- Temporal Integration : Voltage Jumps ( $\Delta V$ ): Each spike adds a discrete "packet of charge" equal to the floating-point weight.
- Voltage climbs in steps (0.039→0.079→0.118).
- Weight vs. Timing: High weights allow the neuron to hit the 1.0 threshold earlier in the simulation.

# Nengo DL (More on it)

- Nengo converts model into tensorflow computation graph.
- There are 2 parts : frontend and backend
- At the front end, it is composed of five objects: Networks (containers), Ensembles (neuron groups), Nodes (input signals), **Connections** (the edges carrying weights), and Probes (for data extraction)
- Internally these are compiled into lower-level Signals (tensor arrays holding simulation state) and Operators (which read signals, compute, and write results). The key operator for weights is DotInc

# DotInc operator (frontend)

- Nengodl uses dotinc operator to multiply weights.
- At every timestep, a layer reads inputs from the previous layer
- Dotinc multiplies the incoming signal by the weights stored in connection. (Matrix-vector multiplication of the input signal by the connection weight matrix)
- Simprocess does synaptic filtering



## Now, incase of SCNNs

- During training it uses a differentiable rate-based approximation of the LIF neuron, because spiking neurons aren't differentiable and gradient descent isn't possible. During inference it switches to actual spiking LIF neurons using those same trained weights. This swap is handled by the framework.
- It takes the weights and assigns them as nengo *connection* objects
- Connections are core components used to define how information moves between different neural ensembles or nodes within a network
- Connections move information (represented as vectors) from a **pre** object to a **post** object.

# Backend

- In the ANN, a weight multiplies a floating-point activation directly. In the SNN, that same weight becomes the transform on a `nengo.connection`. Each incoming spike now contributes weight to the post-neuron's membrane potential via the `DotInc` operator, and a `SimProcess` operator applies synaptic low-pass filtering before passing the signal downstream

- Every pixel of the kernel (5X5) that touches the neuron for convolution becomes a “connection” / synapse.
- Since 5X5 sees 25 pixel values at a time, every neuron is having 25 incoming wires/current and rest of the potential connections become zero.
- So we have a sparse matrix that maps **every pixel** in the input image to **every neuron** in the next layer.

# Quantisation in Neural networks

- To deploy in edge devices like microcontrollers , we need the model to have very less size , so we use quantisation
- edge devices give faster inference
- Generally weights are float64 in a NN
- We convert the float (8 bytes) to a integer to save memory, but this reduces accuracy
- 2 types- post training quantisation and quantisation aware training
- Post training quantisation- train in tf, then convert to tf.lite model but this reduces accuracy, so its better to do the latter
- Quantisation aware training- Quantising a tf model and then training it again **(more information in the pdf of github)**